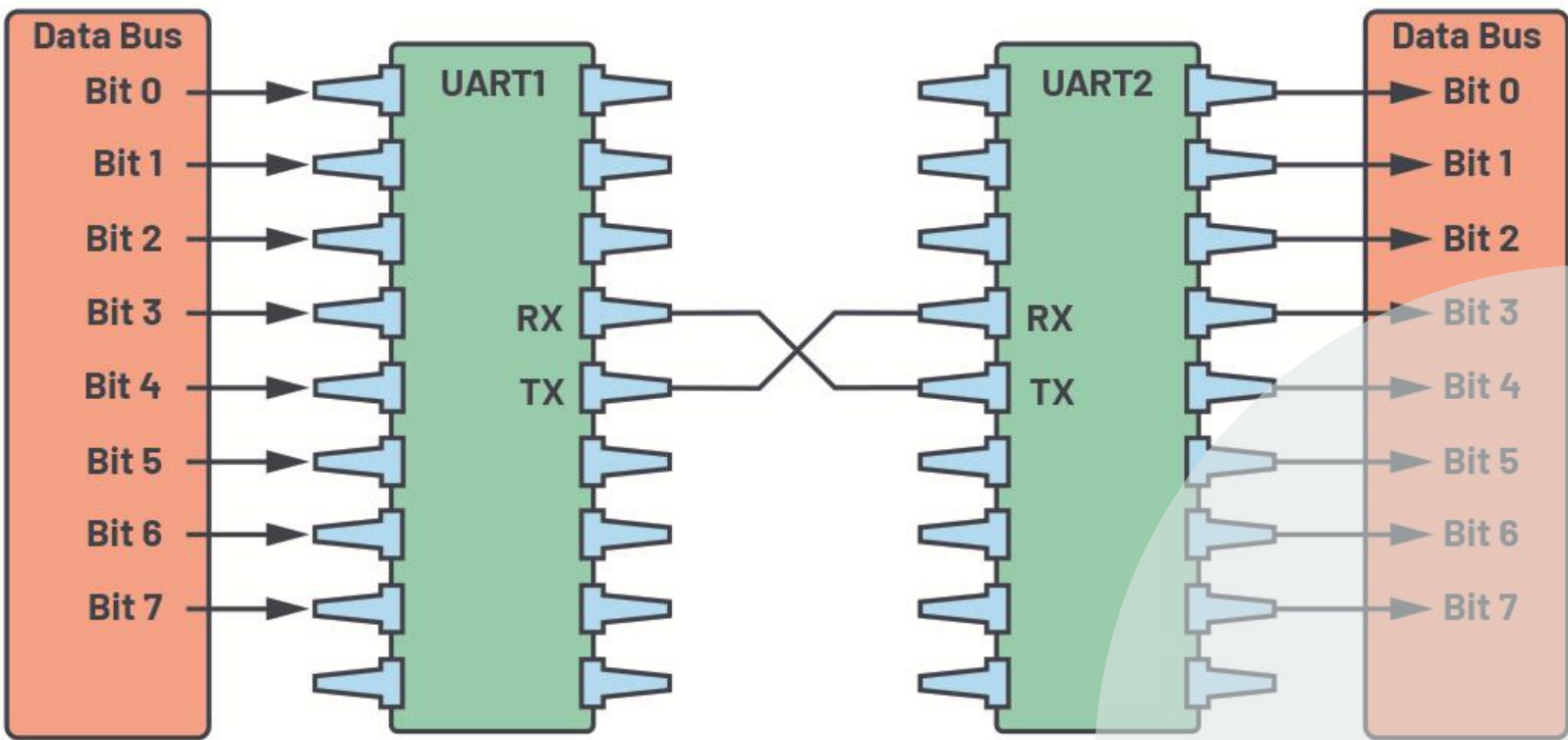




UART

Prepared by:

Arwa Ashraf Kantoush

TABLE OF CONTENTS

1. INTRODUCTION & OVERVIEW

- 1.1 Project Overview & Objectives
- 1.2 UART_RX Architecture Summary
- 1.3 UART_TX Architecture Summary
- 1.4 Operational Features & Frame Transmission Sequence

2. RTL DESIGN & MODELING

2.1 TRANSMITTER

- 2.1.1 Parity Calculator
- 2.1.2 Serializer Shift Register
- 2.1.3 MUX
- 2.1.4 FSM_TX Controller
- 2.1.5 UART_TX Top Integration

2.2 RECEIVER

- 2.2.1 Parity Checker
- 2.2.2 Deserializer Shift Register
- 2.2.3 FSM_RX Controller & Start-Bit Detection
- 2.2.4 UART_RX Top Integration

3. RTL QUALITY CHECK (LINTING)

4. SIMULATION & AUTOMATION

5. ELABORATION FLOW

6. SYNTHESIS FLOW

1. INTRODUCTION & OVERVIEW

1.1 Project Overview & Objectives

This project implements a fully parameterized, cycle-accurate UART core in synthesizable SystemVerilog. The architecture provides independent transmitter (`uart_tx`) and receiver (`uart_rx`) modules featuring modular RTL partitioning, dynamic parity configuration, deterministic FSM control, and error detection.

1.2 UART_RX Architecture Summary

The receiver (`uart_rx`) detects frame boundaries, reconstructs parallel data, and verifies integrity. It integrates a SIPO Deserializer, an FSM_RX controller, and a Parity_Checker. Using a "flag-and-deliver" approach, it outputs the received byte on `o_data` while pulsing `o_valid`, `o_parity_err`, and `o_frame_err` for one clock cycle upon completion.

1.3 UART_TX Architecture Summary

The transmitter (`uart_tx`) converts parallel input into a standard serial bitstream on `o_tx`. Composed of an FSM_TX controller, a PISO Serializer, a Parity_Calculator, and an output MUX, it holds `o_busy` high during transmission, blocks overlapping write attempts, and idles high.

1.4 Operational Features & Frame Transmission Sequence

Each bit period spans exactly one system clock cycle (no oversampling). The transmission frame structure is:

- **Idle:** Serial line held high (`1'b1`).
- **Start Bit:** Driven low (`1'b0`) for one cycle.
- **Data Payload:** `DATA_W` bits (default 8) transmitted LSB-first.
- **Parity Bit (Optional):** Appended when enabled (`i_par_en = 1`), supporting even or odd modes.
- **Stop Bit:** Driven high (`1'b1`) for one cycle.

2. RTL DESIGN & MODELING

2.1 TRANSMITTER

2.1.1 Parity Calculator

Computes and registers the parity bit dynamically upon a valid, non-busy request (`i_valid && i_par_en && !i_busy`) using the reduction operation:

$$o_parity_bit \leq (i_data) \wedge i_par_odd$$

The value remains latched throughout transmission.

2.1.2 Serializer Shift Register

A PISO shift register of width `DATA_W`. It captures `i_data` during load and shifts rightward (LSB-first) on each active `i_ser_en` cycle. A dedicated counter tracks shifted bits and asserts `o_ser_done` concurrently with the final bit.

2.1.3 MUX

A 4-to-1 multiplexer driven by `i_sel` to route the appropriate bit to `o_tx`: Start bit (2'b00), Serial Data (2'b01), Parity bit (2'b10), or Stop bit/Idle (2'b11).

2.1.4 FSM_TX Controller

A Moore FSM traversing `IDLE`, `START`, `DATA`, `PARITY`, and `STOP`. It manages line selection (`o_sel`), shift enable (`o_ser_en`), and `o_busy`. It branches past `PARITY` directly to `STOP` when parity is disabled and rejects write attempts while busy.

2.1.5 UART_TX Top Integration

The top module structuralizes the transmitter by interconnecting the FSM, serializer, parity calculator, and multiplexer into a self-contained entity with clean input/output boundaries.

2.2 RECEIVER

2.2.1 Parity Checker

Combinational logic comparing the received parity bit against the computed reduction of the deserialized data:

```
o_parity_err = (li_par_en)? 0 : (i_parity_bit !== (^i_data)^i_par_odd)? 1 : 0
```

It is clamped to zero when parity is disabled.

2.2.2 Deserializer Shift Register

A SIPO register that shifts incoming **i_rx** bits into the MSB position on **i_shift_en**. An internal counter signals **o_shift_done** once *DATA_W* bits are assembled, resetting cleanly to prevent phase shift on subsequent frames.

2.2.3 FSM_RX Controller & Start-Bit Detection

Governs reception across IDLE, DATA, PARITY, STOP, and PULSE.

- **Start Detection:** Samples **i_rx** only while in IDLE (!**o_busy**), triggering active reception on a low level while masking false start bits during data transfer.
- **Status & Delivery:** Latches parity during PARITY, checks for a valid high stop bit during STOP (registering framing errors if low), and asserts single-cycle pulses on **o_valid** and active error flags during PULSE.

2.2.4 UART_RX Top Integration

Integrates the deserializer, parity checker, and FSM. It coordinates timing so that **o_data** remains stable while **o_valid**, **o_parity_err**, and **o_frame_err** pulse simultaneously.

3. RTL QUALITY CHECK (LINTING)

Static lint analysis was conducted using Siemens Questa Lint following STARC design guidelines. The RTL code achieved zero syntax errors and zero inferred latches. The informational notices regarding duplicate parameter names (DATA_W) are standard due to modular reuse across sub-blocks and have been verified and resolved.

RECEIVER:

The screenshot displays the Siemens Questa Lint 2021.1 interface. The main window shows the RTL code for a module named `uart_rx`. The code defines a parameter `DATA_W = 8` and includes inputs for clock, reset, RX, parity odd, and parity enable. It outputs data, parity error, valid, busy, and frame error signals. The code uses a deserializer and an FSM RX block.

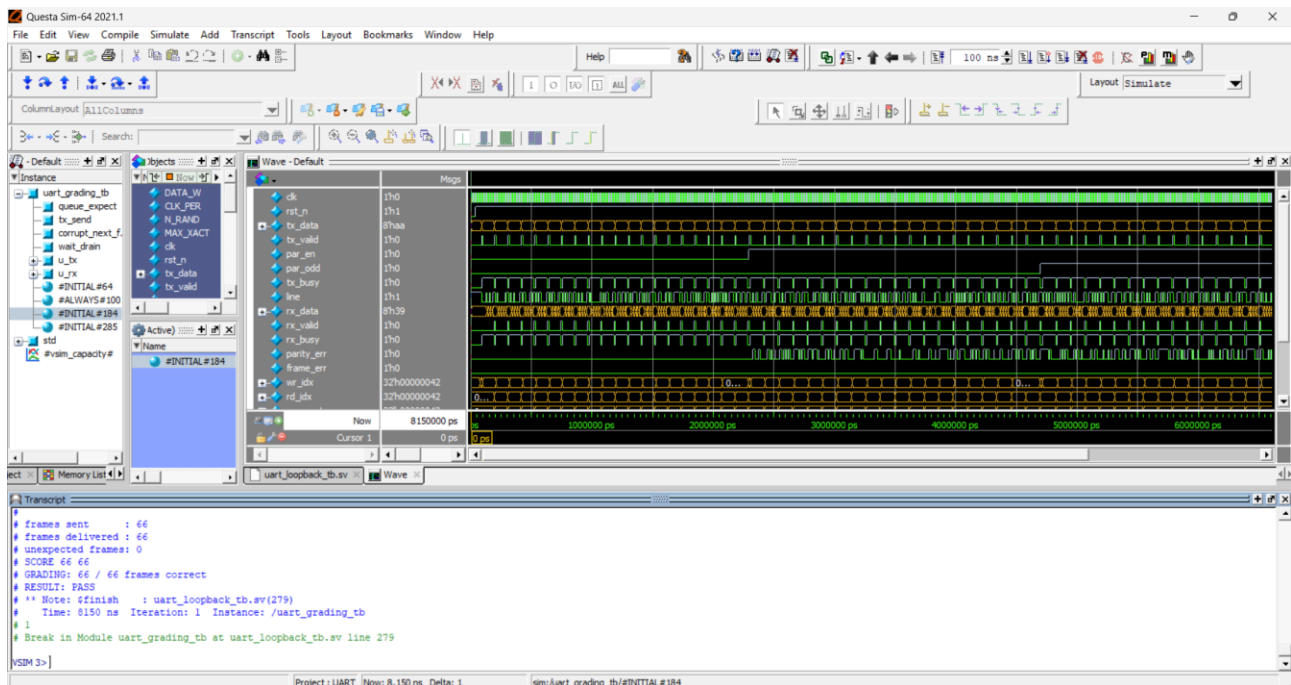
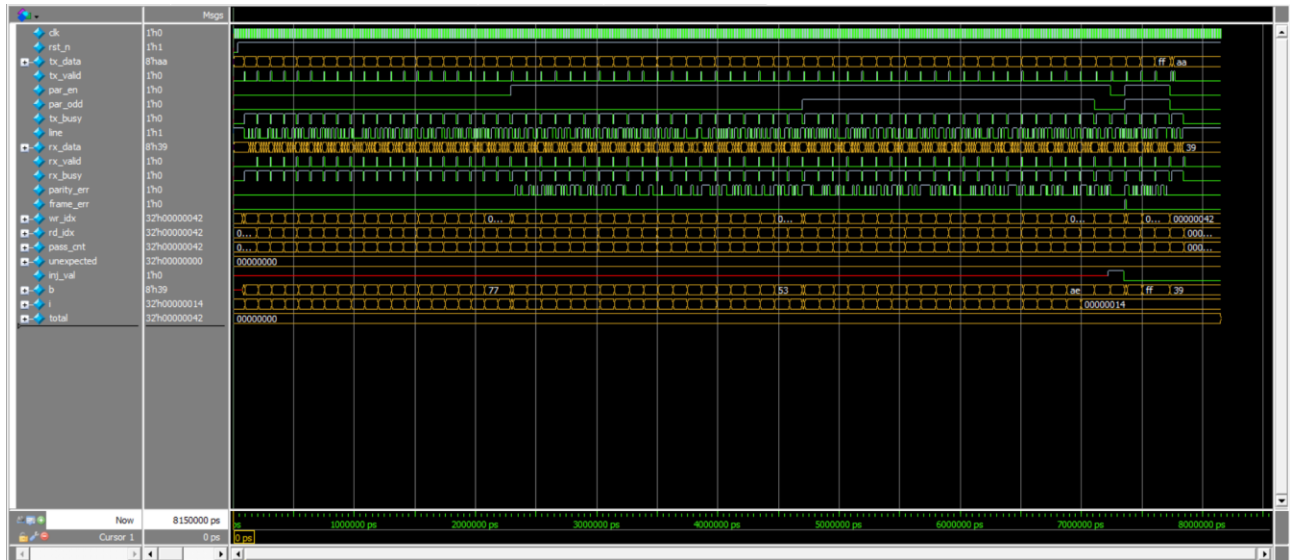
The schematic view on the right shows the internal components: a deserializer, an FSM RX block, and a parity checker. The Lint Checks window at the bottom shows the results of the lint analysis:

Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
Info	Fixed	parameter_name_duplicate		Same parameter name is used in more than one mod...	Deserializer	Nomenclature...	resolved	unassign...	1.1.4.3, 3.2.2.2
Info	Fixed	always_signal_assign_large		Always block has more signal assignments than the s...	FSM_RX	Rtl Design Style	resolved	unassign...	2.6.1.3

[illegible]

4. SIMULATION & AUTOMATION

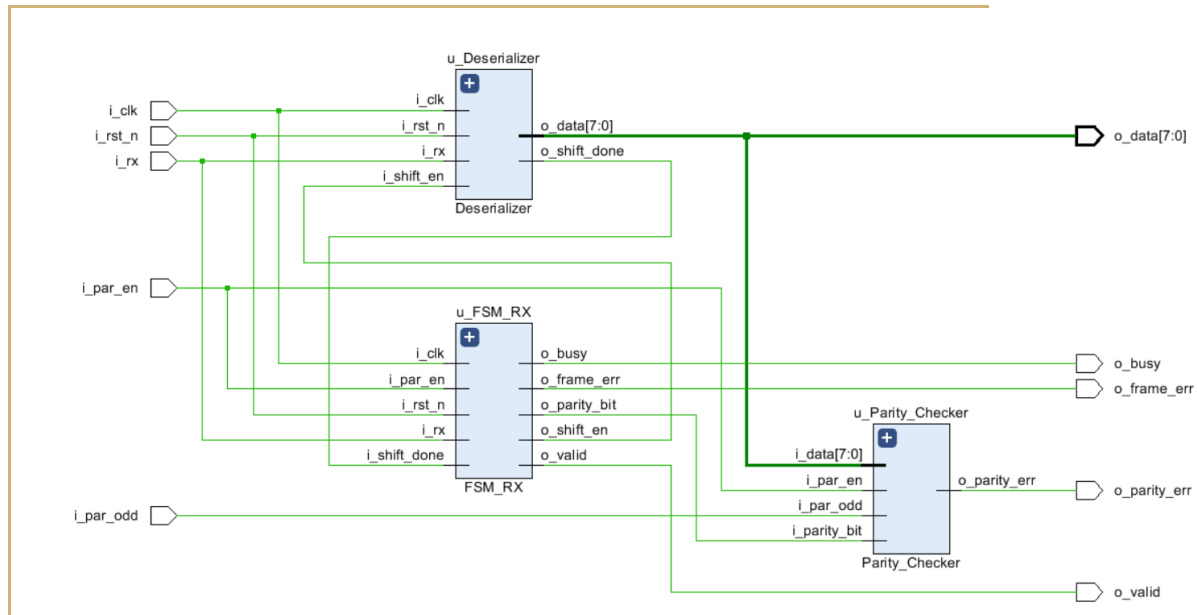
Functional verification was executed using QuestaSim with a self-checking testbench harness. Multiple test cases were evaluated, including parity-disabled modes, even/odd parity transitions, deliberate parity and framing error injections, and back-to-back transactions. The scoreboard confirmed 100% data delivery and correct single-cycle error flag assertion with a clean RESULT: PASS.



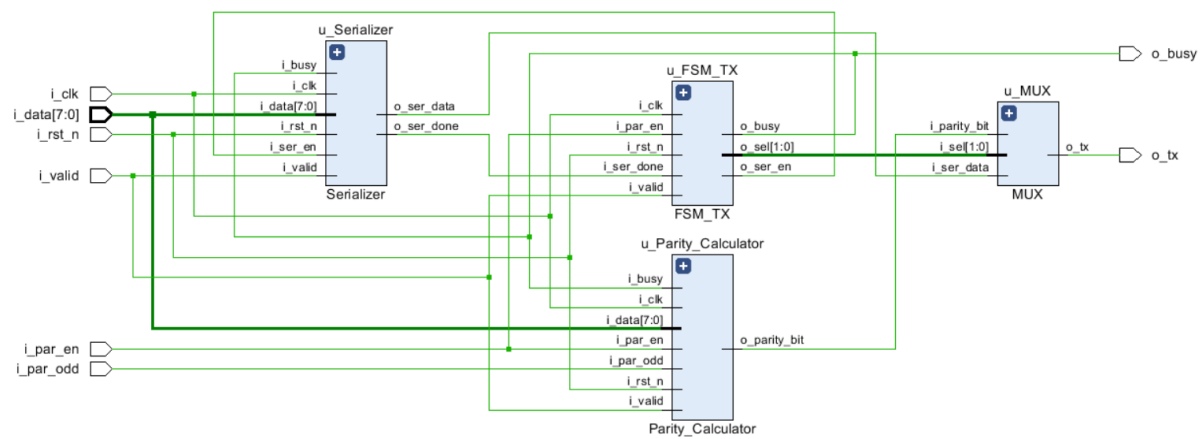
5. ELABORATION FLOW

The elaboration flow interprets the SystemVerilog hierarchy into a technology-independent, generic schematic. The generated netlist demonstrates correct bus widths ($DATA_W = 8$), proper instance interconnections, and clean partitioning between control FSMs and datapath registers for both TX and RX units.

RECEIVER:



TRANSMITTER:

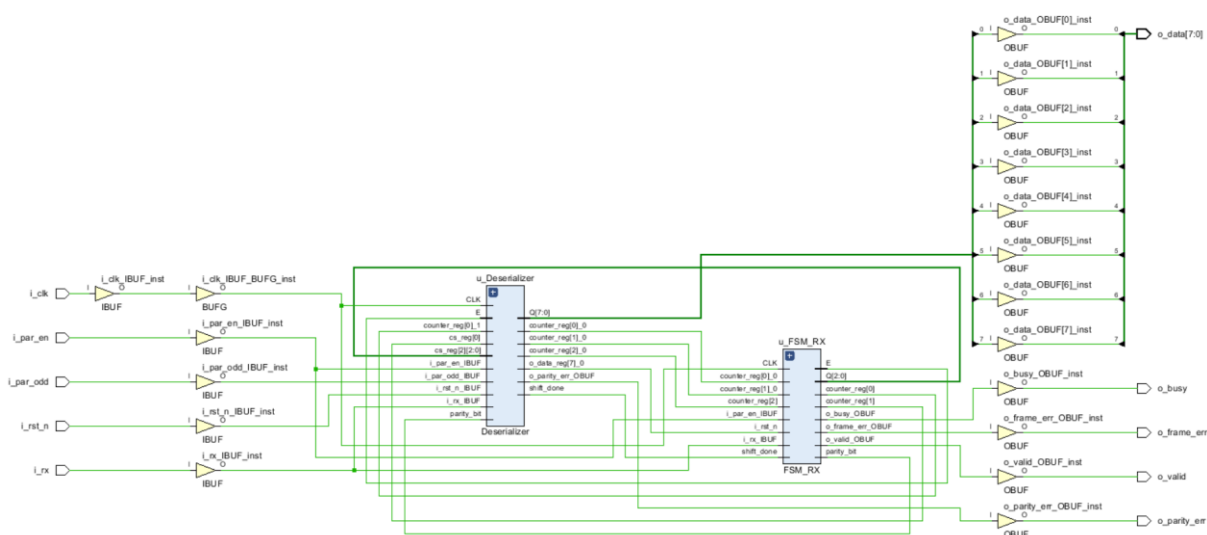


6. SYNTHESIS FLOW

Logic synthesis mapped the design onto physical FPGA primitives (such as Look-Up Tables, Flip-Flops, and I/O Buffers). State machine encodings and timing paths were preserved while optimizing redundant combinational logic.

RECEIVER:

SCHEMATIC:

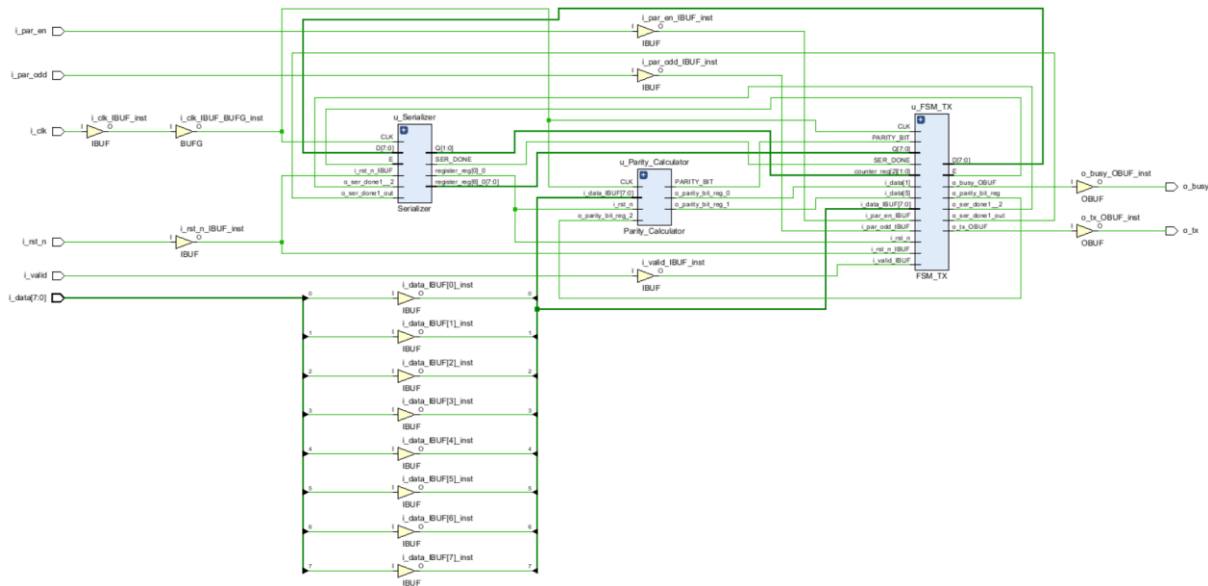


RESOURCE UTILIZATION:

Name	Slice LUTs (303600)	Slice Registers (607200)	Bonded IOB (600)	BUFGCTRL (32)
uart_rx	14	18	17	1
u_Deserializer (Deseri...	7	12	0	0
u_FSM_RX (FSM_RX)	7	6	0	0

TRANSMITTER:

SCHEMATIC:



RESOURCE UTILIZATION:

Name	Slice LUTs (303600)	Slice Registers (607200)	Bonded IOB (600)	BUFGCTRL (32)
uart_tx	23	16	15	1
u_FSM_TX (FSM_TX)	18	3	0	0
u_Parity_Calculator (P...	2	1	0	0
u_Serializer (Serializer)	3	12	0	0